You are experienced excel and python user. prepare exerceses plan to learn PY in excel from scratch to advanced data expoloration and visualisation

Here is a structured exercise plan to go from zero to advanced data exploration and visualization with Python in Excel, assuming you already know Excel well.

Phase 0: Setup and Orientation

- Enable Python in Excel and verify availability:
  - Check your Microsoft 365 version and enable **Insert Python** under **Formulas** or by typing `=PY` in a cell.[1] [2]
- Do 2–3 short "Hello world" exercises:
  - Enter `=PY("print('Hello from Python')")`.
  - Create a simple list or dict and return it to Excel.
- Explore the Python formula editor:
  - Use keyboard shortcuts like `CTRL + Alt + Shift + P` and toggle between Python object and Excel value.[3] [4]

Outcome: Comfortable switching between Excel formulas and Python cells and running basic code in the grid.[5] [1]

Phase 1: Core Python Basics (inside Excel grid)

Focus: Core language, using Excel ranges as inputs, returning simple results.

Exercises (spread over a few sessions):

- Variables and types:
  - Use `=PY("x = 10; y = 3.5; x + y")`.
  - Read a small Excel range via `xl()` and compute min, max, mean.[4]
- Lists, dicts, and loops:
  - Convert a column of numbers to a Python list, loop, and create a transformed list (e.g., 10% uplift).
  - Build a dict mapping product → total quantity.

- Functions:
  - Define a function in the Python cell that cleans a string, then apply it to an Excel column (e.g., trimming, lowercasing).
- Basic error handling:
  - Write a function that handles missing values (`None`) from Excel and replaces them with defaults.

Outcome: Can express simple transformations in Python instead of complex nested Excel formulas.

Phase 2: pandas DataFrames for Tabular Data

Focus: Treat Excel tables as pandas DataFrames and do the bread-and-butter analytics.

- DataFrame basics:
  - Load an Excel table with `xl("Table1", headers=True)` into a DataFrame.[6] [4]
  - Inspect with `.head()`, `.info()`, `.describe()`.
- Column operations:
  - Add calculated columns (e.g., revenue = price × quantity).
  - Convert text dates to proper datetime and derive year/month.
- Filtering and querying:
  - Filter rows (e.g., `sales[sales["Region"] == "EMEA"]`).
  - Use `.query()` for more complex conditions.[4]
- Grouping and aggregation:
  - Group by region/product and compute sum/mean.
  - Create pivot-style summaries with `groupby` and `pivot_table`.
- Saving references:
  - Assign your DataFrame to a named variable (e.g., `sales = xl("Table1[[#All],[..]]", headers=True)`) so you can reuse it in multiple cells.[4]

Outcome: Comfortable using pandas in Excel to replace manual PivotTables and complex SUMIFS combinations.[6] [4]

Phase 3: Data Cleaning and Feature Engineering

Focus: Real-world messy data prep and robust pipelines.

- Missing data:
  - Detect missing with `.isna().sum()`.
  - Impute or drop with `.fillna()` / `.dropna()`.
- Text and categorical data:

    - Clean column names and text fields (strip spaces, unify casing).
    - Create flags and categories (e.g., high_value = Sales > 1000).
- Outlier detection:
    - Use `.describe()` to see outliers and then filter by quantiles or z-scores.[7] [8]
- Reusable cleaning scripts:
    - Write one Python cell that takes the Excel table, applies a sequence of cleaning steps, and outputs a "clean" DataFrame used in subsequent analysis cells.

Outcome: Can build repeatable, auditable cleaning steps instead of ad-hoc Excel fixes.[8] [7] [6]

Phase 4: Exploration & Advanced Analysis

Focus: Deeper descriptive analysis and more complex operations.

- Exploratory statistics:
    - Use `.describe()`, `.value_counts()`, correlations (`df.corr()`).
    - Segment analysis: group by customer/product/time and calculate KPIs.[4]
- Time-series exploration:
    - Resample/aggregate by month/quarter if you have date-based data.
    - Compute rolling metrics (e.g., 3-month rolling average).
- Complex queries:
    - Use `.query()` with string methods and multiple conditions (e.g., color filters, high-value segments).[4]
- Joining and reshaping:
    - Join multiple Excel tables with `merge`.
    - Reshape long ↔ wide with `melt` and `pivot_table`.

Outcome: Able to run rich explorations that would be cumbersome using only Excel formulas.

Phase 5: Visualization with matplotlib / seaborn

Focus: Building charts from Python in Excel and combining with Excel visuals.

- Basic charts:
    - Create line, bar, and scatter plots using `DataFrame.plot()` and matplotlib.[9] [4]
    - Use Python chart output directly in the sheet and compare with native Excel charts.
- Customizing visuals:
    - Customize titles, labels, colors, and legends.
    - Add reference lines and annotations for important thresholds.
- Multi-plot layouts:

    - Produce small multiples or subplots to compare segments.
- Exploratory visual analysis:
    - Visualize distributions (histograms, boxplots).
    - Visualize trends over time and by category.

Outcome: Uses Python charts when needing richer visuals or faster iteration, and Excel charts for final presentation if preferred.[3] [9] [4]

Phase 6: Automation Patterns and "Mini-Apps" in Excel

Focus: Turning scattered analysis into small, maintainable solutions.

- Parameterized analysis:
    - Link Excel input cells (e.g., thresholds, date ranges) to Python scripts that recalculate outputs.[4]
- Reusable report blocks:
    - Build a "report" sheet driven by one or two Python cells that:
        - Load data.
        - Clean and transform.
        - Produce summaries and key charts.
- Performance and structure:
    - Separate heavy transformations into one Python cell; reference its output in lighter downstream cells.
- Optional: Copilot & Python:
    - Experiment with using Copilot in Excel to suggest/insert Python code for a given natural-language analysis task.[10] [11]

Outcome: You have repeatable, parameter-driven workbooks where Python does the heavy lifting and Excel handles layout and distribution.

Suggested Weekly Structure (Example)

Assuming ~5–7 focused hours per week:

- Week 1: Phase 0–1
- Week 2: Finish Phase 1, start Phase 2
- Week 3: Finish Phase 2, Phase 3
- Week 4: Phase 4 (exploration)
- Week 5: Phase 5 (visuals)
- Week 6: Phase 6 (automation + one end-to-end "project" from raw data to dashboard)

If you share what datasets you typically use (credit risk, sales, operations), an adapted exercise set can be drafted around those domains.

⁂

Prepare detailed excersise with credit risk data

Here is a concrete exercise plan using a public credit-risk dataset (e.g. the “Credit Risk Dataset” on Kaggle/OpenML with variables like income, age, loan amount, and a default flag).[21] [22] Assume you have this data in an Excel table named `tbl_credit` with a binary target `loan_status` (0 = good, 1 = default).[23]

1. Data Loading & Sanity Checks

Goal: Bring the portfolio into Python in Excel and understand its structure.

Exercises:

1. Load to pandas:
   - In a Python cell:
     `df = xl("tbl_credit", headers=True)`
     - Display `df.head()`, `df.shape`, `df.dtypes`.
2. Basic quality checks:
   - Compute missing values per column: `df.isna().sum()`.
   - Count good vs defaulted loans: `df["loan_status"].value_counts(normalize=True)`.
3. Simple KPIs:
   - Portfolio size = number of rows.
   - Default rate = mean of `loan_status`.

Deliverable: A small “Overview” section in the sheet (cells) showing portfolio size, default rate, and a short written interpretation.

2. Data Cleaning & Feature Engineering

Goal: Turn raw application/behavioural data into modelling-ready features.[24] [23]

Exercises:

1. Handle missing data:
   - For numeric columns (e.g. `age`, `income`, `loan_amount`), impute with median.
   - For categorical columns (e.g. `home_ownership`, `employment_type`), impute with "Unknown".
2. Create risk-relevant features:
   - Debt-to-income: `dti = total_debt / income`.

- Installment ratio: `inst_ratio = monthly_installment / income`.
- Age buckets: 18–25, 26–35, 36–50, 51+.
- Loan-to-income: `lti = loan_amount / income`.

3. Winsorize / cap outliers:
   - For `income` and `loan_amount`, cap at 1st/99th percentile.

Deliverable: A new DataFrame `df_clean` with all engineered features and a Python cell that prints a concise summary of what was done.

3. Portfolio Segmentation & Descriptive Analysis

Goal: See how default behaves across segments, like in a classic credit-portfolio review.[23] [24]

Exercises:

1. By customer risk drivers:
   - Group by `age_bucket` and compute:
     - Count, average income, default rate.
   - Group by `home_ownership` and compute default rate.
2. By exposure metrics:
   - Bin `loan_amount` into size buckets (e.g. `<10k`, `10–50k`, `>50k`) and compute:
     - Number of accounts.
     - Total exposure.
     - Default rate.
3. Summary table back to Excel:
   - Return a grouped summary (e.g. by `age_bucket` and `loan_amount_bucket`) as an Excel range and format it as a PivotTable-like report.

Deliverable: At least two tables:

- A “By age & default” table.
- A “By loan size & default” table, both displayed as Excel ranges.

4. Visualization of Risk Patterns

Goal: Use Python charts to inspect distributions and risk patterns.[25] [23]

Exercises:

1. Distributions:
   - Histogram of `income` split by `loan_status` (default vs non-default).
   - Boxplot of `loan_amount` by `loan_status`.

2. Segmentation plots:
    - Bar chart of default rate by `age_bucket`.
    - Bar chart of default rate by `home_ownership`.
3. Correlation view:
    - Compute correlation matrix for numeric variables.
    - Plot a heatmap (seaborn) and highlight variables strongly associated with default.

Deliverable: A "Risk Overview" section with 3–4 charts and a short textual comment underneath each (e.g. "higher DTI shows visibly higher default rate").

5. Simple Scorecard-Style Modelling (Logit)

Goal: Build a first supervised PD-style model in Python but orchestrated from Excel.[26] [27] [28]

Exercises:

1. Train–test split:
    - Select features (e.g. `age`, `income`, `dti`, `loan_amount`, `home_ownership` encoded).
    - Use scikit-learn's `train_test_split` to create train/test sets.
2. Logistic regression:
    - Fit `LogisticRegression` to predict `loan_status`.
    - Output model coefficients into an Excel table (feature, coefficient).
3. Model evaluation:
    - Compute accuracy, confusion matrix, precision/recall for the test set (display nicely in a small table).
    - Compute ROC AUC and plot ROC curve.
4. Score & rank:
    - Compute predicted PD for all accounts.
    - Create deciles of predicted PD and compute:
        - Default rate by decile.
        - Share of total exposure by decile.

Deliverable:

- A "Model" sheet showing:
    - Coefficient table.
    - Metrics (accuracy, AUC).
    - PD decile table with default rate and EAD share.

6. Portfolio Risk View: PD Buckets & Expected Loss

Goal: Combine predicted PD with simple LGD/EAD assumptions to produce Expected Loss views.[29] [30]

Assume:

- EAD = `loan_amount`.
- LGD = constant 45% (or a simple rule like 40% for secured, 60% for unsecured).

Exercises:

1. Compute account-level EL:
   - `EL_account = PD * LGD * EAD` for each row in `df_clean`.
2. Portfolio aggregation:
   - Sum total EAD, total EL, and compute average portfolio EL%.
   - Group by PD bucket (e.g. 0–1%, 1–3%, 3–10%, 10%+) and compute:
     - Number of accounts.
     - Total EAD.
     - Total EL.
     - EL% = Total EL / Total EAD.
3. Visualization:
   - Bar chart: EAD by PD bucket.
   - Bar chart: EL% by PD bucket.

Deliverable: A "Portfolio EL" report block with 1–2 tables and 2 charts (EAD and EL% by PD bucket).

7. Scenario / Sensitivity Analysis

Goal: Build a simple "what-if" section, driven by Excel input cells, to stress PD or LGD.[31] [29]

Exercises:

1. Excel input parameters:
   - Create cells for:
     - PD stress factor (e.g. 1.0 baseline, 1.5 mild, 2.0 severe).
     - LGD additive shift (e.g. +5 pp).
2. Python calculation:
   - Read the parameters using `xl()` inside a Python cell.
   - Recalculate EL under:
     - Baseline.

        - Mild stress.
        - Severe stress (e.g. PD × 2, LGD + 5pp).
3. Scenario table:
    - Table with columns: Scenario, Total EAD, Total EL, EL%, change vs baseline.
4. Scenario chart:
    - Bar/column chart showing EL% by scenario.

Deliverable: A "Scenarios" sheet that acts as a mini IFRS-9-style stress test (though simplified).

8. End-to-End "Mini App" in Excel

Goal: Wrap everything into a repeatable workbook template.

Tasks:

1. Structure:
    - Sheet `RawData`: `tbl_credit` (replaceable with future portfolios).
    - Sheet `Inputs`: macroeconomic/parameter cells (LGD, PD factor, thresholds).
    - Sheet `Python_Engine`: a few Python cells:
        - Load & clean data.
        - Build features and model (or read pre-trained coefficients).
        - Compute PD, EL, scenarios.
    - Sheet `Dashboard`: tables and charts linked to the outputs from `Python_Engine`.
2. Refactor:
    - Ensure each Python cell does one clear job:
        - `cell 1`: load & clean.
        - `cell 2`: modelling & PD.
        - `cell 3`: EL & scenarios, returning tidy tables to Excel.
3. Documentation:
    - In a "ReadMe" sheet, describe:
        - Data assumptions.
        - Definition of each key metric.
        - Steps to refresh (e.g. "paste new portfolio into `tbl_credit`, then recalc").

Deliverable: A single workbook where a non-technical colleague can paste new credit-portfolio data and get PD/EL views and stress results with one recalc.

If you share the exact dataset/fields you plan to use (e.g. HMEQ, Kaggle `credit_risk_dataset.csv`), a second pass can give precise Python formulas and column names tailored to that structure.[30] [21] [23]

how to using PY in excel store like module own python functions

Python in Excel does not yet support importing external `.py` modules, but it does allow you to define **reusable functions** in cells and call them from other Python cells in the workbook. You can treat one or a few "library" cells as a local module.[41] [42]

Core idea: define once, call everywhere

- Put your reusable functions in one or a few Python cells (e.g. on a sheet called `py_lib`).
- Other Python cells in the workbook can call those functions as long as they run **after** the cell that defines them.[41]
- Think of those cells as a lightweight in-workbook module.

Example in cell `py_lib!A1`:

```
# =PY(
def clean_income(x):
    if x is None:
        return 0
    return max(0, float(x))

def pd_bucket(pd):
    if pd < 0.01:
        return "0–1%"
    elif pd < 0.03:
        return "1–3%"
    elif pd < 0.10:
        return "3–10%"
    return "10%+"

"functions loaded"
# )
```

Then in another sheet, cell `Sheet1!B2`:

```
# =PY(
value = xl("Sheet1!A2")
clean_income(value)
# )
```

Both `clean_income` and `pd_bucket` are now available to any later Python cell in this workbook.

Practical pattern for a “module sheet”

1. Create sheet `py_lib`:
   - One main Python cell that:
     - Imports core libraries (`pandas`, `numpy`, etc.).
     - Defines all your generic helpers (e.g. credit-risk metrics, bucketing functions, standard charts).
   - Return a simple string like `"library loaded"` so you see it executed.
2. Call order:
   - Make sure Excel calculation runs top-to-bottom / left-to-right so `py_lib` executes before other dependent Python cells.
   - If something “disappears”, recalc or slightly edit the `py_lib` cell to re-run it.
3. Reuse pattern:
   - Only put **definitions** in the library cell (no heavy data operations).
   - In analysis cells, do:
     - `df = xl("tbl_credit", headers=True)`
     - `df["pd_bucket"] = df["pd"].apply(pd_bucket)`

Limitations to be aware of

- No external `.py` or pip installs inside Python in Excel (you are limited to the curated Anaconda environment and in-workbook definitions).[43] [42] [44]
- No true packaging / namespacing; your “module” is just a cell that must run first.
- Scope is per workbook: functions are not shared across workbooks, so you copy the `py_lib` sheet into new workbooks as a template.

Suggested structure for a credit-risk workbook

Create a small in-workbook library with:

- **Data helpers**: `clean_income`, `clean_dpd`, `cap_outliers`, `to_age_band`.
- **Risk metrics**: `expected_loss(pd, lgd, ead)`, `rr_from_lgd(lgd)`.
- **Visualization helpers**: functions that take a DataFrame and create standard plots (default-rate by bucket, EL by segment).

Organize them in `py_lib` and then reuse across all analysis sheets with simple function calls instead of repeating code.[41]

⁂

List typical EDA analysis elements that should by planned on data set

Typical EDA on a credit risk dataset follows a structured sequence to uncover data quality issues, distributions, and risk patterns before modeling.

Dataset Overview

Examine basic structure and target balance.

- Dataset dimensions (rows, columns) and data types per feature.
- Target variable distribution: default rate (e.g., % of loan_status=1), class imbalance check.
- Sample records via head/tail and random sampling.[61] [62]

Univariate Analysis

Inspect individual feature distributions by default status.

- Numeric features: mean, median, std, min/max, histograms/KDEs split by target.
- Categorical features: value counts, mode, bar charts of frequencies by target.
- Missing values: count/sum per column, % missing threshold (e.g., drop >50%).[62] [63]

Bivariate Analysis

Identify relationships with the target and between features.

- Default rate by segments: crosstabs/bar charts for categoricals (age bands, home_ownership, employment).
- Numeric vs target: boxplots, violin plots (e.g., income, DTI, loan_amount by default).
- Correlations: heatmap of numeric features, focus on target correlations.[64] [61]

Outlier & Anomaly Detection

Flag data quality issues.

- Univariate outliers: IQR method, z-scores on numerics (income, age).
- Business rule violations: negative ages/income, illogical DTI >100%.
- Bivariate anomalies: scatter plots (e.g., loan_amount vs income).[63] [62]

Segmentation & Risk Patterns

Core credit risk insights via grouping.

- Default rates by key drivers: age buckets, income quintiles, loan size bands, DTI buckets.
- Exposure views: total EAD (# accounts, sum loan_amount) by segments.
- Time patterns if available: defaults by application month/year.[62] [64]

Data Quality Summary Table

Consolidate findings for quick review.

Check	Metric	Action Needed
Missing %	Income: 5%, Home: 2%	Impute median/mode
Imbalance	Default rate: 8%	SMOTE/undersample
Outliers	Income >99th: 2%	Cap/winsorize
Correlations	DTI vs default: 0.15	Keep as predictor

Visualization Priorities

Produce 8-12 charts total.

- Target balance pie/bar.
- Top 5 numerics: histograms by target.
- Top 5 categoricals: default rate bars.
- Correlation heatmap.
- Key risk driver: default rate heatmap (e.g., age x DTI buckets).[61] [63]

Plan output as one "EDA Report" sheet/section with tables/charts and bullet-point insights (e.g., "DTI>40% drives 25% of defaults"). This feeds directly into cleaning/feature engineering.[64] [62]

⁂

1. https://support.microsoft.com/en-us/office/introduction-to-python-in-excel-55643c2e-ff56-4168-b1ce-9428c8308545
2. https://www.anaconda.com/partners/python-in-excel
3. https://www.youtube.com/watch?v=qEwnlMBaLfc
4. https://www.xda-developers.com/perform-advanced-data-analysis-using-python-in-excel/
5. https://github.com/microsoft/python-in-excel
6. https://www.linkedin.com/learning/paths/advance-your-python-in-excel-skills
7. https://www.xelplus.com/course/python-in-excel/
8. https://digital-skills-jobs.europa.eu/en/opportunities/training/level-python-excel-unlocking-advanced-data-analysis-and-automation
9. https://office-watch.com/2025/python-in-excel-for-microsoft-365-consumers/
10. https://support.microsoft.com/en-gb/office/get-an-advanced-data-analysis-using-copilot-in-excel-with-python-364e4ae9-9343-4d56-952a-5f62b0f70db6
11. https://www.youtube.com/watch?v=7ELGSD-v87g
12. https://www.datacamp.com/tutorial/python-excel-tutorial
13. https://www.youtube.com/watch?v=9zp5bpE7kuc

14. https://www.reddit.com/r/excel/comments/16h85ph/what_are_some_good_resources_to_learn_python_for/
15. https://www.youtube.com/watch?v=ije_fF8SWcw
16. https://www.nobleprog.pl/en/cc/pythonexcel
17. https://learning.anaconda.com/bundles/anaconda-certified-data-analysis-with-python-in-excel
18. https://www.pyxll.com/docs/introduction.html
19. https://github.com/LinkedInLearning/advanced-python-in-excel-data-analysis-and-visualization-4560553
20. https://learn.microsoft.com/en-us/shows/microsoft-at-pycon-us-2024/python-in-excel-live-demo
21. https://www.kaggle.com/datasets/laotse/credit-risk-dataset
22. https://www.openml.org/search?type=data&id=43454
23. https://arthurfeitosasc.github.io/Credit-Risk-Analysis/
24. https://www.listendata.com/2019/08/datasets-for-credit-risk-modeling.html
25. https://lightningchart.com/blog/python/credit-risk-modeling/
26. https://www.almabetter.com/bytes/articles/implementation-of-credit-risk-using-machine-learning-in-python
27. https://blog.jetbrains.com/datalore/2024/08/26/a-complete-guide-to-credit-risk-analysis-with-python-and-datalore-ai/
28. https://www.datacamp.com/courses/credit-risk-modeling-in-python
29. https://www.linkedin.com/pulse/transforming-credit-risk-hands-on-ifrs-9-ecl-model-ravichandran-j9t0c
30. http://www.creditriskanalytics.net/datasets-private2.html
31. https://blog.quantinsti.com/calculating-value-at-risk-in-excel-python/
32. https://www.youtube.com/watch?v=ZY2DxhUo3CY
33. https://sparkco.ai/blog/mastering-credit-rating-models-in-excel-a-deep-dive
34. https://www.kaggle.com/datasets/yasserh/loan-default-dataset
35. https://www.kaggle.com/competitions/predict-loan-default/data
36. https://api.creditriskmonitor.com/APIExamples.aspx
37. https://www.kaggle.com/competitions/credit-default-prediction-ai-big-data
38. https://www.kaggle.com/datasets/nikhil1e9/loan-default
39. https://www.kaggle.com/competitions/bank-loan-default-prediction
40. https://www.youtube.com/watch?v=8uxzXKcrQHg
41. https://www.anaconda.com/blog/5-quick-tips-for-using-python-in-excel
42. https://support.microsoft.com/en-us/office/introduction-to-python-in-excel-55643c2e-ff56-4168-b1ce-9428c8308545
43. https://www.financealliance.io/how-to-use-python-in-excel/
44. https://github.com/microsoft/python-in-excel
45. https://www.youtube.com/watch?v=bbNidIwXZfY
46. https://www.youtube.com/watch?v=qEwnlMBaLfc
47. https://groups.google.com/g/openpyxl-users/c/g-J-E0Djs54

48. https://www.youtube.com/watch?v=9zp5bpE7kuc
49. https://www.datacamp.com/tutorial/python-excel-tutorial
50. https://discuss.python.org/t/memory-persistence-and-variable-passing/82074
51. https://www.reddit.com/r/Python/comments/1de2hkw/my_thoughts_on_python_in_excel/
52. https://www.youtube.com/watch?v=7YS6YDQKFh0
53. https://groups.google.com/g/openpyxl-users/c/OHwe9f4QnWc
54. https://www.softkraft.co/python-excel-automation/
55. https://www.reddit.com/r/learnpython/comments/erl1c0/excel_files_and_python/
56. https://www.reddit.com/r/Python/comments/1hylc50/are_there_any_actual_use_cases_of_python_in_excel/
57. https://support.microsoft.com/en-us/office/get-started-with-python-in-excel-a33fbcbe-065b-41d3-82cf-23d05397f53d
58. https://www.youtube.com/watch?v=FicDbWkofnI
59. https://stackoverflow.com/questions/57574063/avoiding-repetitive-code-in-python-and-reuse-the-existing-code
60. https://stackoverflow.com/questions/43326405/keep-python-global-variables-between-function-calls-in-excel-using-xlwings
61. https://www.kaggle.com/code/anshtanwar/credit-risk-prediction-training-and-eda
62. https://www.blackindata.co.uk/predicting-credit-risk-default-exploratory-data-analysis-eda/
63. https://altair.com/blog/articles/credit-scoring-series-part-three-data-preparation-and-exploratory-data-analysis
64. https://randlow.github.io/posts/machine-learning/kaggle-home-loan-credit-risk-eda/
65. https://fastercapital.com/topics/exploratory-data-analysis-for-credit-risk-modeling.html
66. https://www.finalyse.com/blog/data-management-steps-in-credit-risk-modelling
67. https://www.slideshare.net/slideshow/exploratory-data-analysis-for-credit-risk-assesment/244023673
68. https://arxiv.org/pdf/2407.11976.pdf
69. https://github.com/nafiul-araf/Credit-Risk-Modeling-End-to-End-Project
70. https://fastercapital.com/topics/exploratory-data-analysis-for-credit-risk-analysis.html